EN3160 Assignment 3 — Neural Networks

Name: NAHG Nettikumara
Index number: 230436X
GitHub profile: https://github.com/hasith2003
Repository: https://github.com/hasith2003/Assignment_03

This report covers manual backpropagation on CIFAR-10, LeNet-5 and a modern CNN on MNIST, and two ResNet18 transfer-learning methods for ants and bees.

0. Environment and reproducibility

All experiments ran on CPU with random seed 3160. The environment, library versions, device, and hyperparameters are recorded below. CIFAR-10 and MNIST used their standard training and test splits. The hymenoptera dataset used its supplied training and validation splits. Q1 and Q2 ran for 10 epochs; both Q3 methods ran for 25 epochs. The test split was not used to select Q1 or Q2 checkpoints.

In [1]:
```
from pathlib import Path
import copy
import io
from contextlib import redirect_stdout, redirect_stderr
import csv
import json
import platform
import random
import time
import urllib.request
import zipfile

import numpy as np
import matplotlib.pyplot as plt
import torch
from torch import nn
from torch.utils.data import DataLoader
import torchvision
from torchvision import datasets, models, transforms
from IPython.display import display, Markdown, Image

# Resolve both a kernel launched in CV and one launched in its parent directory.
ROOT = Path.cwd()
if (ROOT / 'CV' / 'en3160_assignment_03.pdf').exists():
    ROOT = ROOT / 'CV'
DATA = ROOT / 'data'
RESULTS = ROOT / 'results'
FIGURES = RESULTS / 'figures'
CHECKPOINTS = ROOT / 'checkpoints'
```

```
for directory in (DATA, RESULTS, FIGURES, CHECKPOINTS):
    directory.mkdir(parents=True, exist_ok=True)
torch.hub.set_dir(str(DATA / 'torch_cache'))

SEED = 3160
DEVICE = torch.device('cpu')
WORKERS = 0  # Reliable in Windows notebooks; increase only if appropriate.

def seed_everything(seed=SEED):
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)

seed_everything()
CONFIG = {
    'seed': SEED,
    'q1': {'epochs': 10, 'batch_size': 50, 'lr': 0.1, 'lr_decay': 0.9},
    'q2': {'epochs': 10, 'batch_size': 64, 'lr': 0.001, 'weight_decay': 0.0},
    'q3': {'epochs': 25, 'batch_size': 4, 'lr': 0.001,
           'momentum': 0.9, 'step_size': 7, 'gamma': 0.1},
}
environment = {'python': platform.python_version(), 'torch': torch.__version__,
               'torchvision': torchvision.__version__, 'numpy': np.__version__,
               'device': str(DEVICE), 'cpu_threads': torch.get_num_threads()}
print(json.dumps(environment, indent=2))
print(json.dumps(CONFIG, indent=2))
(RESULTS / 'configuration.json').write_text(
    json.dumps({'config': CONFIG, 'environment': environment}, indent=2))

def load_quietly(factory, *args, **kwargs):
    # Silence download progress only; exceptions still propagate normally.
    with redirect_stdout(io.StringIO()), redirect_stderr(io.StringIO()):
        return factory(*args, **kwargs)

def make_loader(dataset, batch_size, shuffle=False):
    generator = torch.Generator().manual_seed(SEED)
    return DataLoader(dataset, batch_size=batch_size, shuffle=shuffle,
                      num_workers=WORKERS, generator=generator,
                      pin_memory=False)

def save_rows(name, rows):
    with (RESULTS / f'{name}.csv').open('w', newline='') as handle:
        writer = csv.DictWriter(handle, fieldnames=list(rows[0]))
        writer.writeheader()
        writer.writerows(rows)

def show_table(rows):
    columns = list(rows[0])
    def fmt(value):
        return f'{value:.4f}' if isinstance(value, float) else str(value)
    text = '| ' + ' | '.join(columns) + ' |\n'
    text += '| ' + ' | '.join(['---'] * len(columns)) + ' |\n'
    text += '\n'.join('| ' + ' | '.join(fmt(row[c]) for c in columns) + ' |' for ro
    display(Markdown(text))

def plot_histories(histories, name, heldout='test'):
```

```
    fig, axes = plt.subplots(1, 2, figsize=(13, 4))
    for label, history in histories.items():
        epochs = [r['epoch'] for r in history]
        for ax, metric in zip(axes, ('loss', 'accuracy')):
            ax.plot(epochs, [r[f'train_{metric}'] for r in history], label=f'{label
            ax.plot(epochs, [r[f'{heldout}_{metric}'] for r in history],
                    '--', label=f'{label}: {heldout}')
            ax.set(xlabel='Epoch', ylabel=metric.capitalize())
            ax.grid(alpha=0.25)
            ax.legend(fontsize=8)
    axes[1].set_ylim(0, 1)
    fig.tight_layout()
    fig.savefig(FIGURES / f'{name}.png', dpi=180, bbox_inches='tight')
    display(Image(filename=str(FIGURES / f'{name}.png')))
    plt.close(fig)
```


```
{
  "python": "3.12.1",
  "torch": "2.14.1+cpu",
  "torchvision": "0.29.1+cpu",
  "numpy": "1.26.1",
  "device": "cpu",
  "cpu_threads": 8
}
{
  "seed": 3160,
  "q1": {
    "epochs": 10,
    "batch_size": 50,
    "lr": 0.1,
    "lr_decay": 0.9
  },
  "q2": {
    "epochs": 10,
    "batch_size": 64,
    "lr": 0.001,
    "weight_decay": 0.0
  },
  "q3": {
    "epochs": 25,
    "batch_size": 4,
    "lr": 0.001,
    "momentum": 0.9,
    "step_size": 7,
    "gamma": 0.1
  }
}
```


1. CIFAR-10: a dense network with manual backpropagation

Architecture: **3072 → 100 sigmoid units → 10 logits**. Inputs retain the supplied CIFAR-10 normalization and batch size of 50. The loss is mean multiclass cross-entropy. Regularization is zero, as in the listing. Unlike the listing's tiny initial weights and learning rate, we use fan-

scaled weights and an initial learning rate of 0.1 with the supplied 0.9 per-epoch decay. These are documented implementation choices.

For a batch of size N:

$$H = \sigma(XW_1 + b_1), \quad Z = HW_2 + b_2,$$

$$L = \frac{1}{N} \sum_i \left[\operatorname{logsumexp}(Z_i) - Z_{i,y_i} \right].$$

With $G = (\operatorname{softmax}(Z) - Y)/N$:

$$\nabla W_2 = H^T G, \quad \nabla b_2 = \sum_i G_i,$$

$$D = (GW_2^T) \odot H \odot (1 - H), \quad \nabla W_1 = X^T D, \quad \nabla b_1 = \sum_i D_i.$$

Training uses tensor operations and explicit updates. Autograd is used only in a small independent gradient check, never to train this model.

In [2]:
```
seed_everything()
cifar_transform = transforms.Compose([
    transforms.ToTensor(), transforms.Normalize((0.5,) * 3, (0.5,) * 3)])
# Mirror of the original archive; torchvision verifies its expected MD5.
datasets.CIFAR10.url = ('https://huggingface.co/datasets/liangnanying/'
                        'cifar-10-python/resolve/main/cifar-10-python.tar.gz')
cifar_train = load_quietly(datasets.CIFAR10, DATA, train=True, download=True, trans
cifar_test = load_quietly(datasets.CIFAR10, DATA, train=False, download=True, trans
cifar_train_loader = make_loader(cifar_train, CONFIG['q1']['batch_size'], True)
cifar_train_eval = make_loader(cifar_train, 256)
cifar_test_loader = make_loader(cifar_test, 256)
print('CIFAR-10 sizes:', len(cifar_train), len(cifar_test))

class ManualMLP:
    def __init__(self, din=3072, hidden=100, classes=10,
                 device=DEVICE, dtype=torch.float32):
        self.params = {
            'W1': torch.randn(din, hidden, device=device, dtype=dtype) / din**0.5,
            'b1': torch.zeros(hidden, device=device, dtype=dtype),
            'W2': torch.randn(hidden, classes, device=device, dtype=dtype) / hidden
            'b2': torch.zeros(classes, device=device, dtype=dtype),
        }

    def forward(self, x):
        p = self.params
        h = torch.sigmoid(x @ p['W1'] + p['b1'])
        return h @ p['W2'] + p['b2'], h

    def loss_and_grads(self, x, labels):
        logits, h = self.forward(x)
        n = len(labels)
        loss = (torch.logsumexp(logits, dim=1)
                - logits[torch.arange(n, device=x.device), labels]).mean()
```

```
        g = logits.softmax(dim=1)
        g[torch.arange(n, device=x.device), labels] -= 1
        g /= n
        dh = (g @ self.params['W2'].T) * h * (1 - h)
        grads = {'W1': x.T @ dh, 'b1': dh.sum(0),
                 'W2': h.T @ g, 'b2': g.sum(0)}
        return loss, grads

    @torch.no_grad()
    def step(self, grads, lr):
        for key, parameter in self.params.items():
            parameter.add_(grads[key], alpha=-lr)

# Check every manual derivative on a small double-precision example.
seed_everything()
tiny = ManualMLP(7, 5, 3, device='cpu', dtype=torch.float64)
x_check = torch.randn(4, 7, dtype=torch.float64)
y_check = torch.tensor([0, 1, 2, 1])
manual_loss, manual_grads = tiny.loss_and_grads(x_check, y_check)
reference = {k: v.clone().requires_grad_() for k, v in tiny.params.items()}
h_check = torch.sigmoid(x_check @ reference['W1'] + reference['b1'])
reference_loss = nn.functional.cross_entropy(
    h_check @ reference['W2'] + reference['b2'], y_check)
reference_grads = torch.autograd.grad(reference_loss, tuple(reference.values()))
torch.testing.assert_close(manual_loss, reference_loss)
for key, grad in zip(reference, reference_grads):
    torch.testing.assert_close(manual_grads[key], grad, rtol=1e-7, atol=1e-9)
print('Manual loss and all four gradients agree with autograd.')
```

```
CIFAR-10 sizes: 50000 10000
Manual loss and all four gradients agree with autograd.
```

In [3]:

```
@torch.no_grad()
def evaluate_manual(model, loader):
    total_loss, correct, count = 0.0, 0, 0
    for images, labels in loader:
        images, labels = images.to(DEVICE), labels.to(DEVICE)
        logits, _ = model.forward(images.flatten(1))
        total_loss += nn.functional.cross_entropy(logits, labels, reduction='sum').
        correct += (logits.argmax(1) == labels).sum().item()
        count += labels.numel()
    return {'loss': total_loss / count, 'accuracy': correct / count}

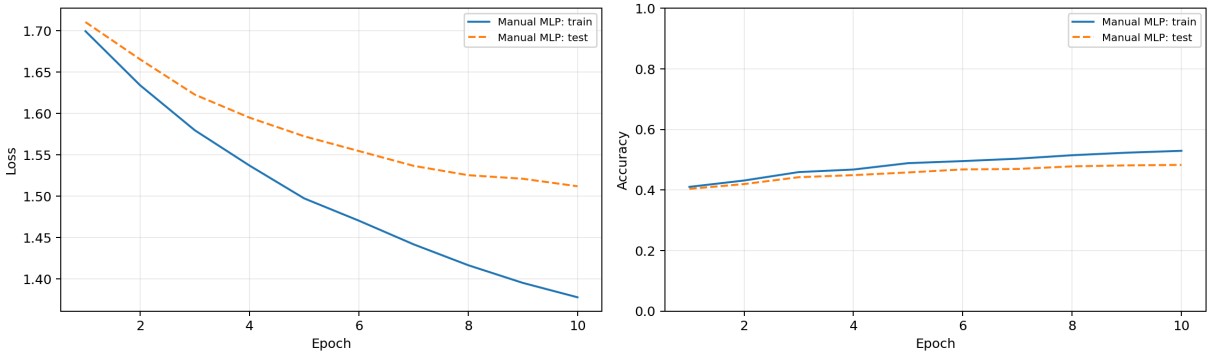
seed_everything()
manual_model = ManualMLP()
q1_history = []
start = time.perf_counter()
lr = CONFIG['q1']['lr']
for epoch in range(1, CONFIG['q1']['epochs'] + 1):
    for images, labels in cifar_train_loader:
        images, labels = images.to(DEVICE), labels.to(DEVICE)
        loss, grads = manual_model.loss_and_grads(images.flatten(1), labels)
        if not torch.isfinite(loss) or not all(torch.isfinite(g).all() for g in gra
            raise FloatingPointError('Non-finite loss or gradients; inspect the lea
        manual_model.step(grads, lr)
    train = evaluate_manual(manual_model, cifar_train_eval)
```

```
    test = evaluate_manual(manual_model, cifar_test_loader)
    row = {'epoch': epoch, 'lr': lr, 'train_loss': train['loss'],
           'train_accuracy': train['accuracy'], 'test_loss': test['loss'],
           'test_accuracy': test['accuracy']}
    q1_history.append(row)
    save_rows('q1_manual_mlp', q1_history)
    print(f"Epoch {epoch:02d}: train={train['accuracy']:.2%}, test={test['accuracy'
    lr *= CONFIG['q1']['lr_decay']
q1_seconds = time.perf_counter() - start
torch.save({k: v.cpu() for k, v in manual_model.params.items()}, CHECKPOINTS / 'q1_
show_table([q1_history[-1]])
plot_histories({'Manual MLP': q1_history}, 'q1_learning_curves')
print(f'Training plus per-epoch evaluation: {q1_seconds:.1f} seconds')
```

```
Epoch 01: train=41.01%, test=40.36%
Epoch 02: train=43.12%, test=41.94%
Epoch 03: train=45.90%, test=44.21%
Epoch 04: train=46.74%, test=44.89%
Epoch 05: train=48.85%, test=45.78%
Epoch 06: train=49.53%, test=46.78%
Epoch 07: train=50.32%, test=46.93%
Epoch 08: train=51.47%, test=47.79%
Epoch 09: train=52.30%, test=48.11%
Epoch 10: train=52.92%, test=48.27%
```

epoch	lr	train_loss	train_accuracy	test_loss	test_accuracy
10	0.0387	1.3780	0.5292	1.5122	0.4827

```
<Figure size 1300x400 with 2 Axes>
Training plus per-epoch evaluation: 247.7 seconds
```



Q1 interpretation

After 10 epochs, training accuracy was **52.92%** and test accuracy was **48.27%**. Training cross-entropy fell from 1.6996 to 1.3780; test cross-entropy fell from 1.7107 to 1.5122. The final accuracy gap was 4.65 percentage points. Both losses decreased over the run, so the curves do not show a late rise in held-out loss indicating pronounced overfitting within these 10 epochs.

The moderate training accuracy suggests that optimization and/or model capacity remain limiting factors; this run alone does not isolate their contributions. Flattening removes the explicit spatial layout used by convolutional networks, and this model has only one hidden

layer of 100 sigmoid units. The manually derived loss and all four parameter gradients passed the double-precision autograd check. The network uses fan-scaled initialization and an initial learning rate of 0.1 instead of the listing's tiny weights and 2e-6 learning rate. This was intended to produce measurable learning with the new hidden layer and cross-entropy loss within 10 epochs. The final loss and accuracy curves show that learning occurred. No test-driven checkpoint selection was performed.

2. MNIST: LeNet-5 versus a small modern CNN

Both models receive MNIST padded to 32×32 and normalized identically. Both train for 10 epochs with Adam, learning rate 0.001, batch size 64, no weight decay, and no learning-rate scheduler or augmentation. Seeds and shuffled batch order are reset for each experiment. These settings are fixed before the final comparison.

LeNet-5 uses the common dense-layer adaptation: 6/16 convolutional channels, tanh, average pooling, and dense layers of 120/84/10. It is not a literal reproduction of the original sparse connections and RBF output. The modern CNN has exactly three convolutions, each with batch normalization and ReLU, plus dropout in its head.

All reported training/test metrics are evaluated **after each epoch in evaluation mode**, without dropout and with batch-normalization running statistics. Accuracy is a fraction; mean loss is sample-weighted. Epoch-10 models supply the confusion matrices; test results are not used for model selection.

In [4]:
```
mnist_transform = transforms.Compose([
    transforms.Pad(2), transforms.ToTensor(), transforms.Normalize((0.1307,), (0.30
mnist_train = load_quietly(datasets.MNIST, DATA, train=True, download=True, transfo
mnist_test = load_quietly(datasets.MNIST, DATA, train=False, download=True, transfo
mnist_train_eval = make_loader(mnist_train, 256)
mnist_test_loader = make_loader(mnist_test, 256)
print('MNIST sizes:', len(mnist_train), len(mnist_test))

class LeNet5(nn.Module):
    def __init__(self):
        super().__init__()
        self.layers = nn.Sequential(
            nn.Conv2d(1, 6, 5), nn.Tanh(), nn.AvgPool2d(2),
            nn.Conv2d(6, 16, 5), nn.Tanh(), nn.AvgPool2d(2),
            nn.Flatten(), nn.Linear(16 * 5 * 5, 120), nn.Tanh(),
            nn.Linear(120, 84), nn.Tanh(), nn.Linear(84, 10))

    def forward(self, x):
        return self.layers(x)

class SmallCNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.layers = nn.Sequential(
            nn.Conv2d(1, 32, 3, padding=1, bias=False), nn.BatchNorm2d(32),
```

```
            nn.ReLU(), nn.MaxPool2d(2),
            nn.Conv2d(32, 64, 3, padding=1, bias=False), nn.BatchNorm2d(64),
            nn.ReLU(), nn.MaxPool2d(2),
            nn.Conv2d(64, 128, 3, padding=1, bias=False), nn.BatchNorm2d(128),
            nn.ReLU(), nn.AdaptiveAvgPool2d(1), nn.Flatten(),
            nn.Dropout(0.3), nn.Linear(128, 10))

    def forward(self, x):
        return self.layers(x)

def parameter_counts(model):
    return {'parameters': sum(p.numel() for p in model.parameters()),
            'trainable': sum(p.numel() for p in model.parameters() if p.requires_gr

for cls in (LeNet5, SmallCNN):
    model = cls().eval()
    assert model(torch.zeros(2, 1, 32, 32)).shape == (2, 10)
    print(cls.__name__, parameter_counts(model))

def train_one_epoch(model, loader, optimizer, frozen_backbone=False):
    model.train()
    if frozen_backbone:
        # Fix both backbone weights and BatchNorm running statistics.
        model.eval()
        model.fc.train()
    for images, labels in loader:
        images, labels = images.to(DEVICE), labels.to(DEVICE)
        optimizer.zero_grad(set_to_none=True)
        loss = nn.functional.cross_entropy(model(images), labels)
        if not torch.isfinite(loss):
            raise FloatingPointError('Non-finite training loss')
        loss.backward()
        optimizer.step()

@torch.no_grad()
def evaluate(model, loader, collect=False):
    model.eval()
    total_loss, correct, count = 0.0, 0, 0
    targets, predictions = [], []
    for images, labels in loader:
        images, labels = images.to(DEVICE), labels.to(DEVICE)
        logits = model(images)
        predicted = logits.argmax(1)
        total_loss += nn.functional.cross_entropy(logits, labels, reduction='sum').
        correct += (predicted == labels).sum().item()
        count += labels.numel()
        if collect:
            targets.append(labels.cpu())
            predictions.append(predicted.cpu())
    result = {'loss': total_loss / count, 'accuracy': correct / count}
    if collect:
        result.update(targets=torch.cat(targets), predictions=torch.cat(predictions
    return result

def fit(model, train_loader, train_eval_loader, heldout_loader, optimizer,
        epochs, name, heldout='test', scheduler=None, frozen_backbone=False,
```

```
        select_best=False):
    history, best_accuracy, best_epoch = [], -1.0, None
    start = time.perf_counter()
    for epoch in range(1, epochs + 1):
        used_lr = optimizer.param_groups[0]['lr']
        train_one_epoch(model, train_loader, optimizer, frozen_backbone)
        train_metrics = evaluate(model, train_eval_loader)
        heldout_metrics = evaluate(model, heldout_loader)
        row = {'epoch': epoch, 'lr': used_lr,
               'train_loss': train_metrics['loss'], 'train_accuracy': train_metrics
               f'{heldout}_loss': heldout_metrics['loss'],
               f'{heldout}_accuracy': heldout_metrics['accuracy']}
        history.append(row)
        save_rows(name, history)
        print(f"{name} {epoch:02d}/{epochs}: train={train_metrics['accuracy']:.2%},
              f"{heldout}={heldout_metrics['accuracy']:.2%}")
        if select_best and heldout_metrics['accuracy'] > best_accuracy:
            best_accuracy, best_epoch = heldout_metrics['accuracy'], epoch
            torch.save(model.state_dict(), CHECKPOINTS / f'{name}_best.pt')
        if scheduler is not None:
            scheduler.step()
    seconds = time.perf_counter() - start
    torch.save(model.state_dict(), CHECKPOINTS / f'{name}_final.pt')
    metadata = {'seconds_train_and_evaluation': seconds,
                'best_epoch': best_epoch,
                'best_accuracy': best_accuracy if select_best else None,
                **parameter_counts(model)}
    (RESULTS / f'{name}_metadata.json').write_text(json.dumps(metadata, indent=2))
    return history, metadata

def confusion_matrix_plot(targets, predictions, names, title, filename):
    k = len(names)
    matrix = torch.bincount(targets * k + predictions, minlength=k*k).reshape(k, k)
    fig, ax = plt.subplots(figsize=(7, 6))
    im = ax.imshow(matrix, cmap='Blues')
    fig.colorbar(im, ax=ax)
    ax.set(xticks=range(k), yticks=range(k), xticklabels=names, yticklabels=names,
           xlabel='Predicted label', ylabel='True label', title=title)
    for i in range(k):
        for j in range(k):
            ax.text(j, i, str(matrix[i, j]), ha='center', va='center', fontsize=8,
                    color='white' if matrix[i, j] > matrix.max()/2 else 'black')
    fig.tight_layout()
    fig.savefig(FIGURES / f'{filename}.png', dpi=180, bbox_inches='tight')
    np.savetxt(RESULTS / f'{filename}.csv', matrix, delimiter=',', fmt='%d')
    display(Image(filename=str(FIGURES / f'{filename}.png')))
    plt.close(fig)
    return matrix
```

```
MNIST sizes: 60000 10000
LeNet5 {'parameters': 61706, 'trainable': 61706}
SmallCNN {'parameters': 94186, 'trainable': 94186}
```

In [5]:

```
q2_histories, q2_metadata, q2_matrices = {}, {}, {}
for label, cls in [('LeNet5', LeNet5), ('SmallCNN', SmallCNN)]:
    seed_everything()
```

```
    model = cls().to(DEVICE)
    train_loader = make_loader(mnist_train, CONFIG['q2']['batch_size'], True)
    optimizer = torch.optim.Adam(model.parameters(), lr=CONFIG['q2']['lr'],
                                 weight_decay=CONFIG['q2']['weight_decay'])
    history, metadata = fit(model, train_loader, mnist_train_eval, mnist_test_loade
                            optimizer, CONFIG['q2']['epochs'], f'q2_{label}')
    q2_histories[label], q2_metadata[label] = history, metadata
    result = evaluate(model, mnist_test_loader, collect=True)
    q2_matrices[label] = confusion_matrix_plot(
        result['targets'], result['predictions'], list(range(10)),
        f'{label}: MNIST test, epoch 10', f'q2_{label}_confusion')

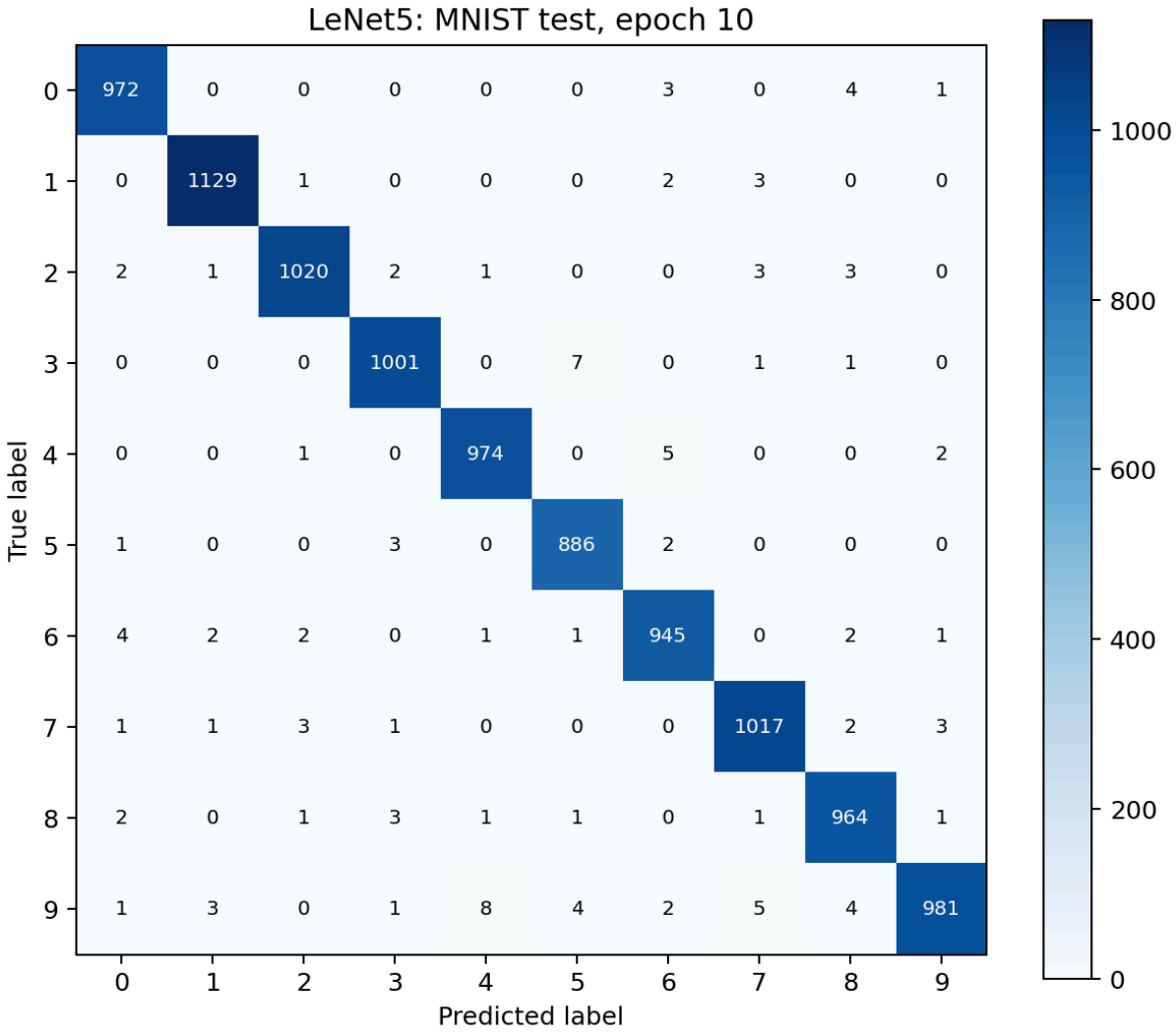
plot_histories(q2_histories, 'q2_model_comparison')
q2_summary = [dict(model=label, **history[-1], **q2_metadata[label])
              for label, history in q2_histories.items()]
show_table([{
    'Model': row['model'],
    'Train accuracy': f"{row['train_accuracy']:.2%}",
    'Test accuracy': f"{row['test_accuracy']:.2%}",
    'Train loss': f"{row['train_loss']:.4f}",
    'Test loss': f"{row['test_loss']:.4f}",
    'Parameters': f"{row['parameters']:,}",
    'Time (min)': f"{row['seconds_train_and_evaluation']/60:.2f}",
} for row in q2_summary])
save_rows('q2_summary', q2_summary)

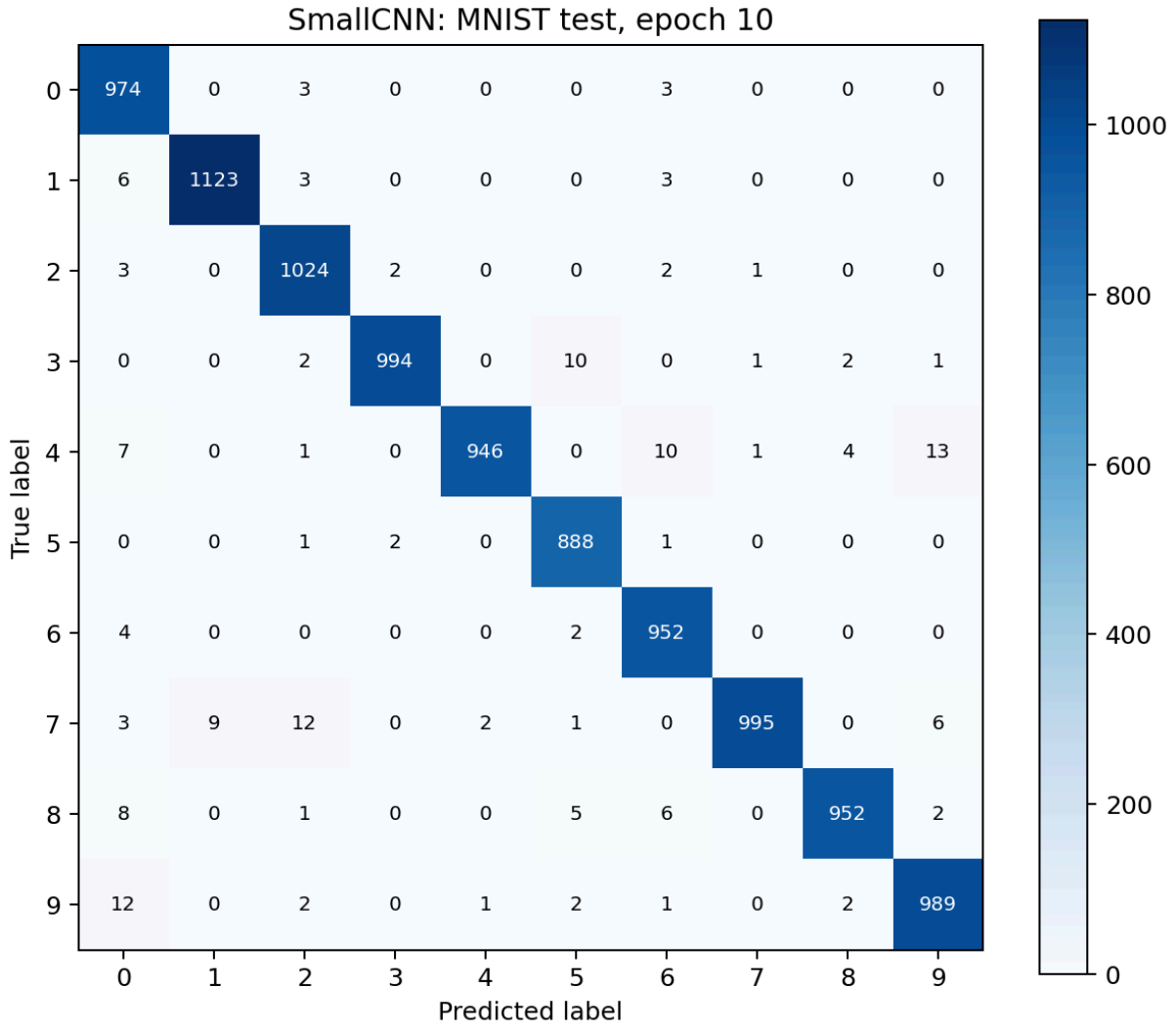
for label, matrix in q2_matrices.items():
    errors = matrix.copy()
    np.fill_diagonal(errors, 0)
    pairs = np.argsort(errors.ravel())[::-1][:5]
    print(label, 'largest off-diagonal counts (true, predicted, count):',
          [(int(i//10), int(i%10), int(errors.ravel()[i])) for i in pairs if errors
```

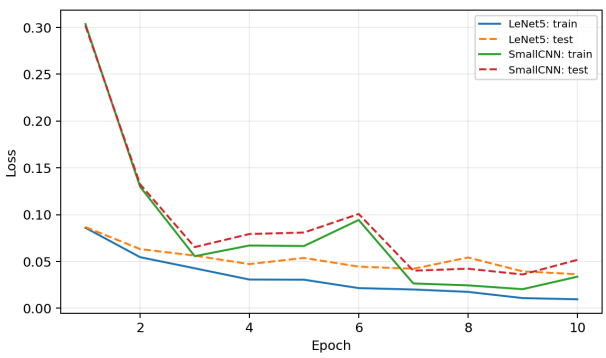
```
q2_LeNet5 01/10: train=97.43%, test=97.46%
q2_LeNet5 02/10: train=98.33%, test=97.89%
q2_LeNet5 03/10: train=98.66%, test=98.24%
q2_LeNet5 04/10: train=99.07%, test=98.56%
q2_LeNet5 05/10: train=99.09%, test=98.29%
q2_LeNet5 06/10: train=99.34%, test=98.54%
q2_LeNet5 07/10: train=99.35%, test=98.69%
q2_LeNet5 08/10: train=99.44%, test=98.37%
q2_LeNet5 09/10: train=99.67%, test=98.72%
q2_LeNet5 10/10: train=99.73%, test=98.89%
q2_SmallCNN 01/10: train=89.72%, test=89.79%
q2_SmallCNN 02/10: train=95.98%, test=95.67%
q2_SmallCNN 03/10: train=98.40%, test=98.13%
q2_SmallCNN 04/10: train=97.97%, test=97.44%
q2_SmallCNN 05/10: train=97.90%, test=97.44%
q2_SmallCNN 06/10: train=96.75%, test=96.72%
q2_SmallCNN 07/10: train=99.23%, test=98.81%
q2_SmallCNN 08/10: train=99.30%, test=98.75%
q2_SmallCNN 09/10: train=99.41%, test=98.89%
q2_SmallCNN 10/10: train=98.91%, test=98.37%
```

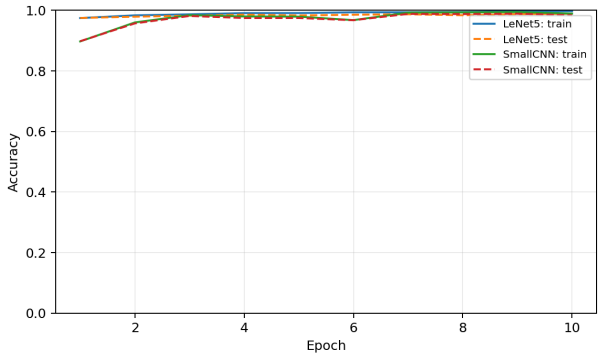
Model	Train accuracy	Test accuracy	Train loss	Test loss	Parameters	Time (min)
LeNet-5	99.73%	98.89%	0.0096	0.0363	61,706	5.77
Modern CNN	98.91%	98.37%	0.0337	0.0517	94,186	14.24

```
LeNet5 largest off-diagonal counts (true, predicted, count): [(9, 4, 8), (3, 5, 7),
(4, 6, 5), (9, 7, 5), (6, 0, 4)]
SmallCNN largest off-diagonal counts (true, predicted, count): [(4, 9, 13), (9, 0, 1
2), (7, 2, 12), (4, 6, 10), (3, 5, 10)]
```









Q2 comparison

At epoch 10, LeNet-5 achieved **99.73% training accuracy** and **98.89% test accuracy**, with training/test losses of 0.0096/0.0363. The modern CNN achieved **98.91% training accuracy** and **98.37% test accuracy**, with losses of 0.0337/0.0517. LeNet-5's test accuracy was 0.52 percentage points higher in this fixed-budget run.

LeNet-5 had 61,706 parameters versus 94,186 for the modern CNN. Training plus per-epoch evaluation took 5.77 and 14.24 minutes respectively on this CPU. These timings exclude data downloads and depend on the local machine's workload. The modern architecture's

additional convolutional computation did not deliver higher final accuracy under these common settings. Its learning curves fluctuate more, and its epoch-10 accuracy is below epoch 9; the prescribed epoch-10 checkpoint is retained rather than choosing a checkpoint using test performance.

The final training-minus-test accuracy gaps were 0.84 percentage points for LeNet-5 and 0.54 for the modern CNN. A smaller gap alone does not establish a better model when training and test accuracies are also lower. Batch normalization, dropout, pooling, activations, and channel counts differ together, so this comparison cannot isolate any one feature.

LeNet-5's largest confusion was true 9 predicted as 4 (8 images), followed by 3 predicted as 5 (7). The modern CNN's largest was 4 predicted as 9 (13), followed by 9 predicted as 0 and 7 predicted as 2 (12 each). The matrices each contain all 10,000 test images and their diagonals agree with the reported accuracies.

This is one seeded run. The 0.52-percentage-point final test gap is small relative to the models' nearly 99% accuracies, so it does not establish a general ranking. The modern CNN's test accuracy falls at epochs 4–6 before recovering; changes in batch-normalization running statistics or optimization could contribute, but this experiment does not isolate a cause. MNIST's high accuracy also leaves little room for an architectural gain under the chosen training budget.

3. Transfer learning: ants versus bees

Use the existing `train` and `val` splits, with training augmentation, 224×224 crops, and ImageNet normalization. A separate deterministic view of the training split is used for reported training metrics. Both methods start independently from `ResNet18_Weights.IMAGENET1K_V1`, with the same seeded two-class head initialization.

- **Fine-tuning:** all parameters train.
- **Feature extraction:** only the final fully connected layer trains. Backbone parameters and batch-normalization running statistics stay fixed.

Both use SGD (learning rate 0.001, momentum 0.9), batch size 4, 25 epochs, and StepLR (step 7, gamma 0.1). Q3 does not prescribe an epoch count; 25 follows the tutorial. We report final and best validation results separately. Validation selects the best checkpoint, so these are validation estimates, not independent test estimates.

In [6]:
```
hym_root = DATA / 'hymenoptera_data'
if not ((hym_root / 'train').is_dir() and (hym_root / 'val').is_dir()):
    archive = DATA / 'hymenoptera_data.zip'
    urllib.request.urlretrieve(
        'https://download.pytorch.org/tutorial/hymenoptera_data.zip', archive)
    with zipfile.ZipFile(archive) as bundle:
        # Reject archive entries that would escape the data directory.
```

```
        for entry in bundle.infolist():
            destination = (DATA / entry.filename).resolve()
            if not destination.is_relative_to(DATA.resolve()):
                raise ValueError('Unsafe archive path')
        bundle.extractall(DATA)

imagenet_mean = (0.485, 0.456, 0.406)
imagenet_std = (0.229, 0.224, 0.225)
hym_train_transform = transforms.Compose([
    transforms.RandomResizedCrop(224), transforms.RandomHorizontalFlip(),
    transforms.ToTensor(), transforms.Normalize(imagenet_mean, imagenet_std)])
hym_eval_transform = transforms.Compose([
    transforms.Resize(256), transforms.CenterCrop(224), transforms.ToTensor(),
    transforms.Normalize(imagenet_mean, imagenet_std)])
hym_train = datasets.ImageFolder(hym_root / 'train', transform=hym_train_transform)
hym_train_deterministic = datasets.ImageFolder(hym_root / 'train', transform=hym_ev
hym_val = datasets.ImageFolder(hym_root / 'val', transform=hym_eval_transform)
assert hym_train.class_to_idx == hym_val.class_to_idx
assert len(hym_train.classes) == 2
hym_train_eval = make_loader(hym_train_deterministic, 8)
hym_val_loader = make_loader(hym_val, 8)
print('Classes:', hym_train.class_to_idx)
for label, dataset in [('train', hym_train), ('val', hym_val)]:
    print(label, len(dataset), dict(zip(dataset.classes,
          np.bincount(dataset.targets, minlength=2).tolist())))

def build_resnet(feature_extraction=False):
    model = load_quietly(models.resnet18, weights=models.ResNet18_Weights.IMAGENET1
    if feature_extraction:
        for parameter in model.parameters():
            parameter.requires_grad_(False)
    # Reset independently so both methods receive the same new head.
    torch.manual_seed(SEED)
    model.fc = nn.Linear(model.fc.in_features, 2)
    return model.to(DEVICE)
```

```
Classes: {'ants': 0, 'bees': 1}
train 244 {'ants': 123, 'bees': 121}
val 153 {'ants': 70, 'bees': 83}
```

In [7]:

```
q3_histories, q3_metadata, q3_summary = {}, {}, []
for label, frozen in [('fine_tuning', False), ('feature_extraction', True)]:
    seed_everything()
    model = build_resnet(feature_extraction=frozen)
    train_loader = make_loader(hym_train, CONFIG['q3']['batch_size'], True)
    optimizer = torch.optim.SGD(
        [p for p in model.parameters() if p.requires_grad],
        lr=CONFIG['q3']['lr'], momentum=CONFIG['q3']['momentum'])
    scheduler = torch.optim.lr_scheduler.StepLR(
        optimizer, step_size=CONFIG['q3']['step_size'], gamma=CONFIG['q3']['gamma']
    history, metadata = fit(
        model, train_loader, hym_train_eval, hym_val_loader, optimizer,
        CONFIG['q3']['epochs'], f'q3_{label}', heldout='val', scheduler=scheduler,
        frozen_backbone=frozen, select_best=True)
    q3_histories[label], q3_metadata[label] = history, metadata
    q3_summary.append(dict(method=label, final_train_accuracy=history[-1]['train_ac
```

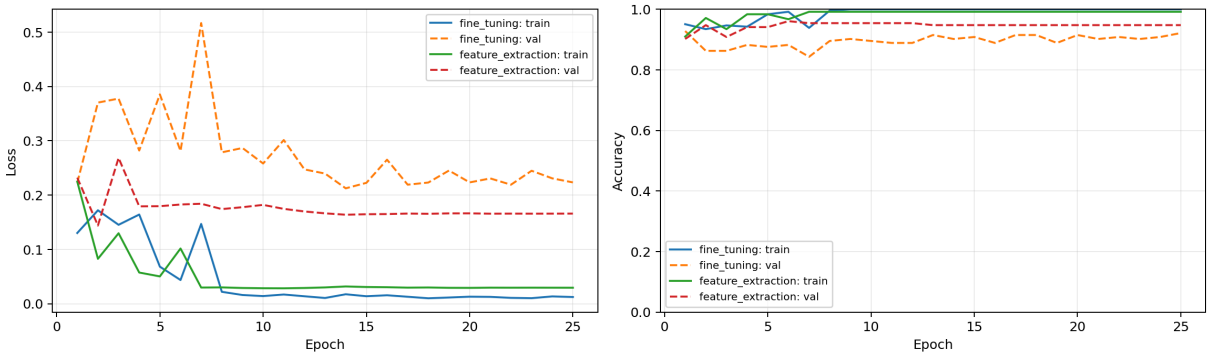
```
                            final_val_accuracy=history[-1]['val_accuracy'], **metada

plot_histories(q3_histories, 'q3_transfer_comparison', heldout='val')
show_table([{
    'Method': row['method'].replace('_', ' ').title(),
    'Final train': f"{row['final_train_accuracy']:.2%}",
    'Final validation': f"{row['final_val_accuracy']:.2%}",
    'Best validation': f"{row['best_accuracy']:.2%}",
    'Best epoch': row['best_epoch'],
    'Trainable': f"{row['trainable']:,}",
    'Time (min)': f"{row['seconds_train_and_evaluation']/60:.2f}",
} for row in q3_summary])
save_rows('q3_summary', q3_summary)
```

```
q3_fine_tuning 01/25: train=95.08%, val=92.81%
q3_fine_tuning 02/25: train=93.44%, val=86.27%
q3_fine_tuning 03/25: train=94.67%, val=86.27%
q3_fine_tuning 04/25: train=94.26%, val=88.24%
q3_fine_tuning 05/25: train=98.36%, val=87.58%
q3_fine_tuning 06/25: train=99.18%, val=88.24%
q3_fine_tuning 07/25: train=93.85%, val=84.31%
q3_fine_tuning 08/25: train=99.59%, val=89.54%
q3_fine_tuning 09/25: train=100.00%, val=90.20%
q3_fine_tuning 10/25: train=100.00%, val=89.54%
q3_fine_tuning 11/25: train=100.00%, val=88.89%
q3_fine_tuning 12/25: train=100.00%, val=88.89%
q3_fine_tuning 13/25: train=100.00%, val=91.50%
q3_fine_tuning 14/25: train=100.00%, val=90.20%
q3_fine_tuning 15/25: train=100.00%, val=90.85%
q3_fine_tuning 16/25: train=100.00%, val=88.89%
q3_fine_tuning 17/25: train=100.00%, val=91.50%
q3_fine_tuning 18/25: train=100.00%, val=91.50%
q3_fine_tuning 19/25: train=100.00%, val=88.89%
q3_fine_tuning 20/25: train=100.00%, val=91.50%
q3_fine_tuning 21/25: train=100.00%, val=90.20%
q3_fine_tuning 22/25: train=100.00%, val=90.85%
q3_fine_tuning 23/25: train=100.00%, val=90.20%
q3_fine_tuning 24/25: train=100.00%, val=90.85%
q3_fine_tuning 25/25: train=100.00%, val=92.16%
q3_feature_extraction 01/25: train=90.98%, val=90.20%
q3_feature_extraction 02/25: train=97.13%, val=94.77%
q3_feature_extraction 03/25: train=93.44%, val=90.85%
q3_feature_extraction 04/25: train=98.36%, val=94.12%
q3_feature_extraction 05/25: train=98.36%, val=94.12%
q3_feature_extraction 06/25: train=96.72%, val=96.08%
q3_feature_extraction 07/25: train=99.18%, val=95.42%
q3_feature_extraction 08/25: train=99.18%, val=95.42%
q3_feature_extraction 09/25: train=99.18%, val=95.42%
q3_feature_extraction 10/25: train=99.18%, val=95.42%
q3_feature_extraction 11/25: train=99.18%, val=95.42%
q3_feature_extraction 12/25: train=99.18%, val=95.42%
q3_feature_extraction 13/25: train=99.18%, val=94.77%
q3_feature_extraction 14/25: train=99.18%, val=94.77%
q3_feature_extraction 15/25: train=99.18%, val=94.77%
q3_feature_extraction 16/25: train=99.18%, val=94.77%
q3_feature_extraction 17/25: train=99.18%, val=94.77%
```

```
q3_feature_extraction 18/25: train=99.18%, val=94.77%
q3_feature_extraction 19/25: train=99.18%, val=94.77%
q3_feature_extraction 20/25: train=99.18%, val=94.77%
q3_feature_extraction 21/25: train=99.18%, val=94.77%
q3_feature_extraction 22/25: train=99.18%, val=94.77%
q3_feature_extraction 23/25: train=99.18%, val=94.77%
q3_feature_extraction 24/25: train=99.18%, val=94.77%
q3_feature_extraction 25/25: train=99.18%, val=94.77%
```

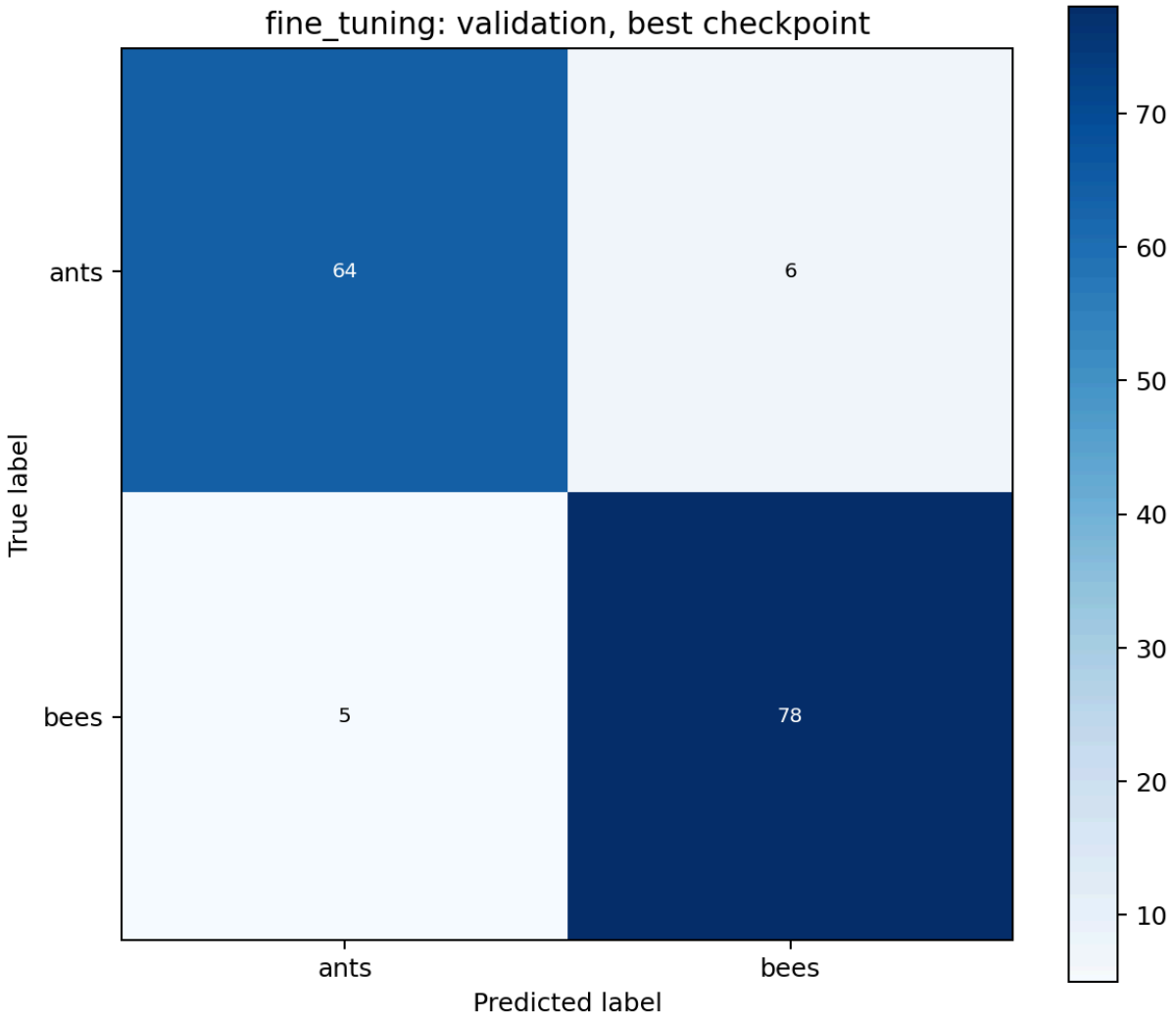
Method	Final train	Final validation	Best validation	Best epoch	Trainable	Time (min)
Fine-tuning	100.00%	92.16%	92.81%	1	11,177,538	9.51
Feature extraction	99.18%	94.77%	96.08%	6	1,026	5.79

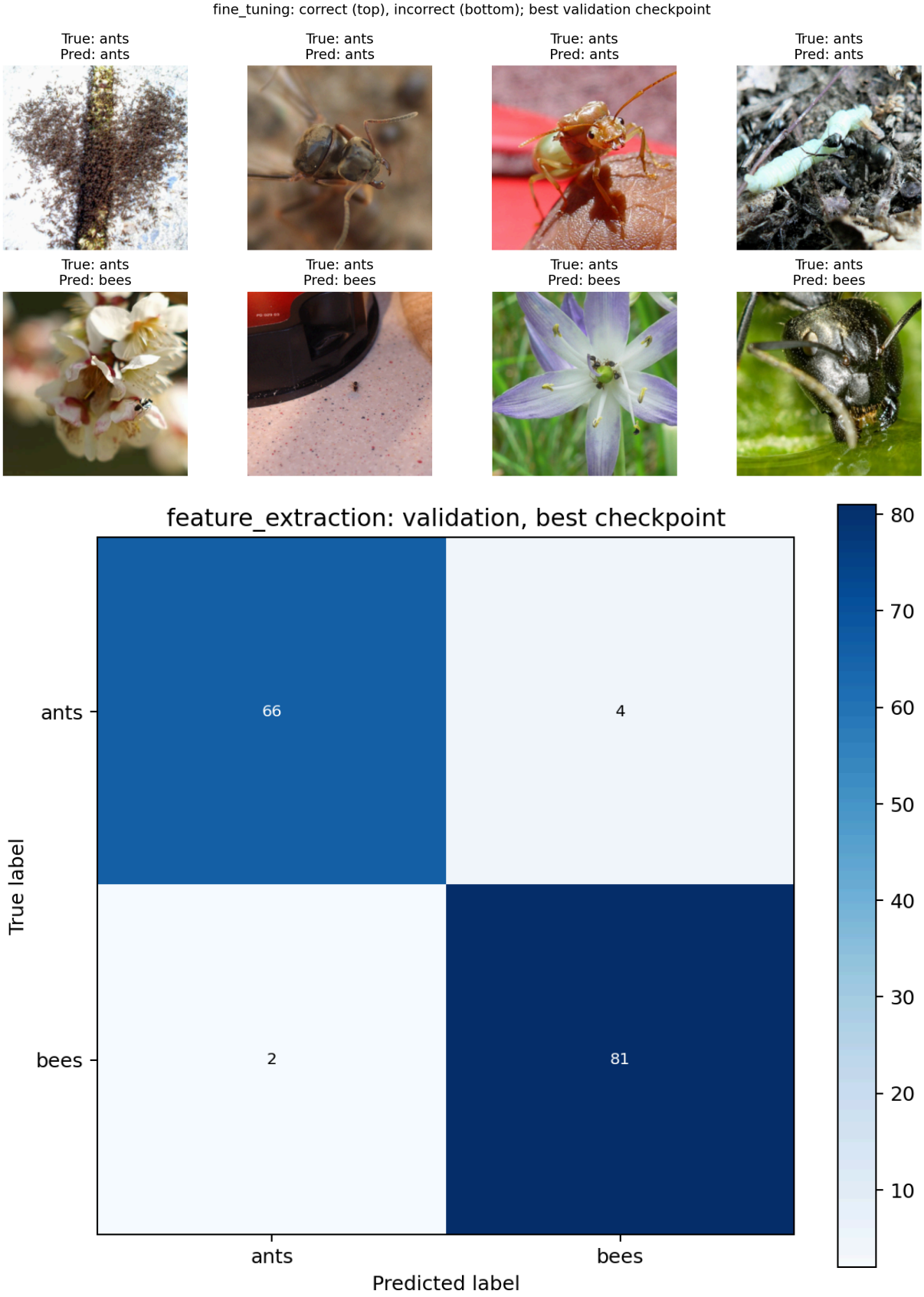


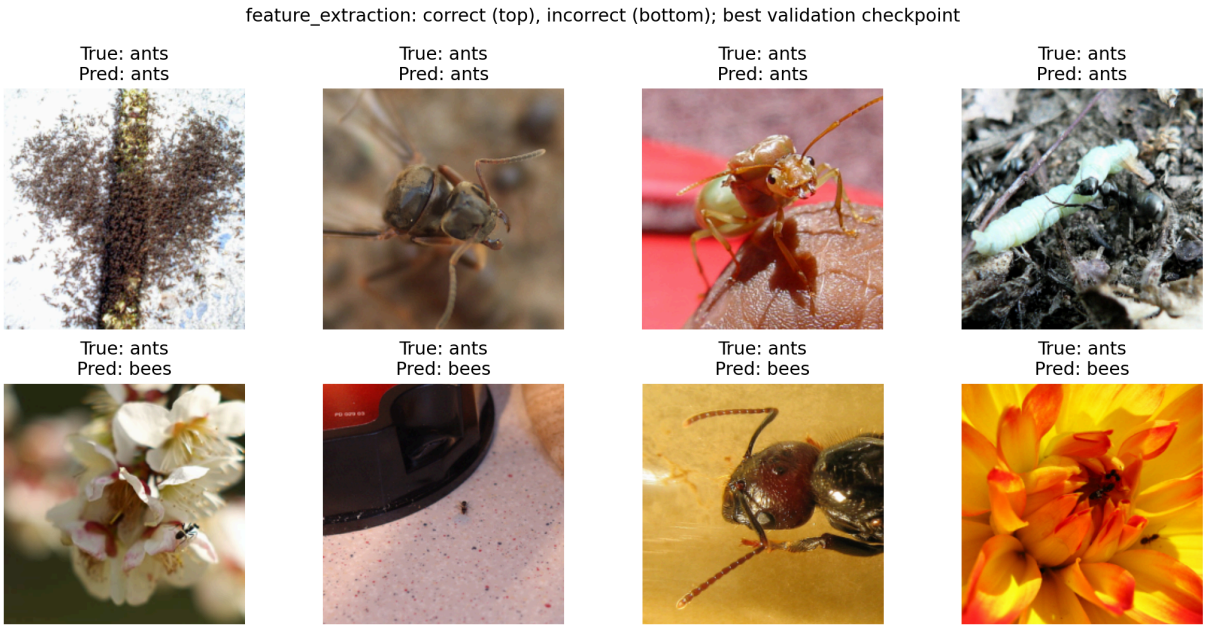
In [8]:
```python
@torch.no_grad()
def show_prediction_examples(model, dataset, name, max_each=4):
    model.eval()
    correct_examples, wrong_examples = [], []
    for image, target in dataset:
        predicted = model(image.unsqueeze(0).to(DEVICE)).argmax(1).item()
        group = correct_examples if predicted == target else wrong_examples
        if len(group) < max_each:
            group.append((image, target, predicted))
        if len(correct_examples) >= max_each and len(wrong_examples) >= max_each:
            break
    fig, axes = plt.subplots(2, max_each, figsize=(3 * max_each, 6), squeeze=False)
    for row, examples in enumerate((correct_examples, wrong_examples)):
        for column, ax in enumerate(axes[row]):
            ax.axis('off')
            if column < len(examples):
                image, target, predicted = examples[column]
                image = image.permute(1, 2, 0).numpy()
                image = np.clip(image * np.array(imagenet_std) + np.array(imagenet_
                ax.imshow(image)
                ax.set_title(f'True: {dataset.classes[target]}\nPred: {dataset.clas
            else:
                ax.set_title('No further examples')
    fig.suptitle(f'{name}: correct (top), incorrect (bottom); best validation check
    fig.tight_layout()
    fig.savefig(FIGURES / f'q3_{name}_examples.png', dpi=180, bbox_inches='tight')
    display(Image(filename=str(FIGURES / f'q3_{name}_examples.png')))
    plt.close(fig)
```

```python
for label, frozen in [('fine_tuning', False), ('feature_extraction', True)]:
    model = build_resnet(feature_extraction=frozen)
    model.load_state_dict(torch.load(CHECKPOINTS / f'q3_{label}_best.pt',
                                     map_location=DEVICE, weights_only=True))
    result = evaluate(model, hym_val_loader, collect=True)
    print(label, 'best checkpoint validation accuracy:', f"{result['accuracy']:.2%}
    confusion_matrix_plot(result['targets'], result['predictions'], hym_val.classes
                          f'{label}: validation, best checkpoint', f'q3_{label}_con
    show_prediction_examples(model, hym_val, label)
```

```
fine_tuning best checkpoint validation accuracy: 92.81%
feature_extraction best checkpoint validation accuracy: 96.08%
```




fine_tuning: correct (top), incorrect (bottom); best validation checkpoint
True: ants
Pred: ants
True: ants
Pred: ants
True: ants
Pred: ants
True: ants
Pred: ants
True: ants
Pred: bees
True: ants
Pred: bees
True: ants
Pred: bees
True: ants
Pred: bees
feature_extraction: validation, best checkpoint
66
4
2
81
ants
bees
True label
ants
bees
Predicted label
80
70
60
50
40
30
20
10



Q3 interpretation

- **Fine tuning:** final training/validation accuracy 100.00%/92.16%; final training/validation loss 0.0123/0.2233. Best validation accuracy was 92.81% at epoch 1. Trainable parameters: 11,177,538. Training plus evaluation: 9.51 minutes.
- **Feature extraction:** final training/validation accuracy 99.18%/94.77%; final training/validation loss 0.0294/0.1659. Best validation accuracy was 96.08% at epoch 6. Trainable parameters: 1,026. Training plus evaluation: 5.79 minutes.

Feature extraction's best validation accuracy was 3.27 percentage points higher than fine-tuning's. Its final validation accuracy was 94.77%, compared with 92.16% for fine-tuning. Fine-tuning took 1.64 times as long including evaluation. In this run, the fixed ImageNet representation was already useful for the two-class task; updating every backbone parameter did not produce the highest observed validation score. This is a result for these settings and this small split, rather than evidence that feature extraction always wins.

Fine-tuning updates both the pretrained visual representation and the new classifier. Feature extraction retains the pretrained representation and learns only the two-class linear head; the backbone's batch-normalization statistics are fixed as well. The trainable parameter counts and measured times quantify this computational trade-off. Both methods use 25 epochs, the same preprocessing, and independent copies of the same ImageNet-pretrained weights. Reported training metrics use deterministic crops, whereas optimization uses random augmentation.

The displayed confusion matrices and correct/incorrect examples use each method's **best validation checkpoint**, whereas the final-epoch results above use epoch 25. Because validation accuracy selects the checkpoint, its best score is not an unbiased independent test estimate. With only 153 validation images, one changed prediction changes accuracy by

about 0.65 percentage points. A single seeded run does not quantify run-to-run variability; interpret small differences cautiously. The training/validation curves and misclassified images provide context for the numerical comparison.

Fine-tuning reached its highest validation accuracy in epoch 1, while training accuracy later reached 100%. This gap is consistent with overfitting on the small dataset. Updating all pretrained weights at learning rate 0.001 may also affect early validation performance, but these results do not isolate that effect. The StepLR schedule reduced the fine-tuning learning rate to about 1e-6 after epoch 21; subsequent changes in the curves were small.

The example images show different kinds of difficult ant images: one ant is tiny within a flower scene, while another fills the crop and is still classified as a bee. This suggests that object size alone does not explain the errors. Background and framing may matter, but these pictures cannot prove which features drove a prediction. The displayed examples were selected in dataset order, so they show ants; the confusion matrices evaluate both ants and bees.

4. Conclusions

The manual network learned a useful CIFAR-10 classifier with verified explicit gradients. For MNIST, LeNet-5 was both faster and slightly more accurate than this modern CNN under the shared 10-epoch settings. The transfer-learning experiments demonstrate the measured accuracy and computation trade-off between adapting the entire network and fitting only a classifier. Accuracy values across CIFAR-10, MNIST, and ants/bees should not be compared as a controlled architecture study because their datasets and tasks differ.